

Knowledge Garden — Project Specification

Track 2: Development & Local Deployment of Private AI Agents AMD AI DevMaster Hackathon 2026-07

1. Application scenario

Knowledge Garden is a fully-private learning companion Agent that helps an individual learner turn any study material into a structured, verified learning loop. It is the "personal tutor" every self-learner wants: it extracts the key knowledge from what you upload, quizzes you with the Feynman technique, grades your recall against the source, and schedules reviews on the Ebbinghaus forgetting curve.

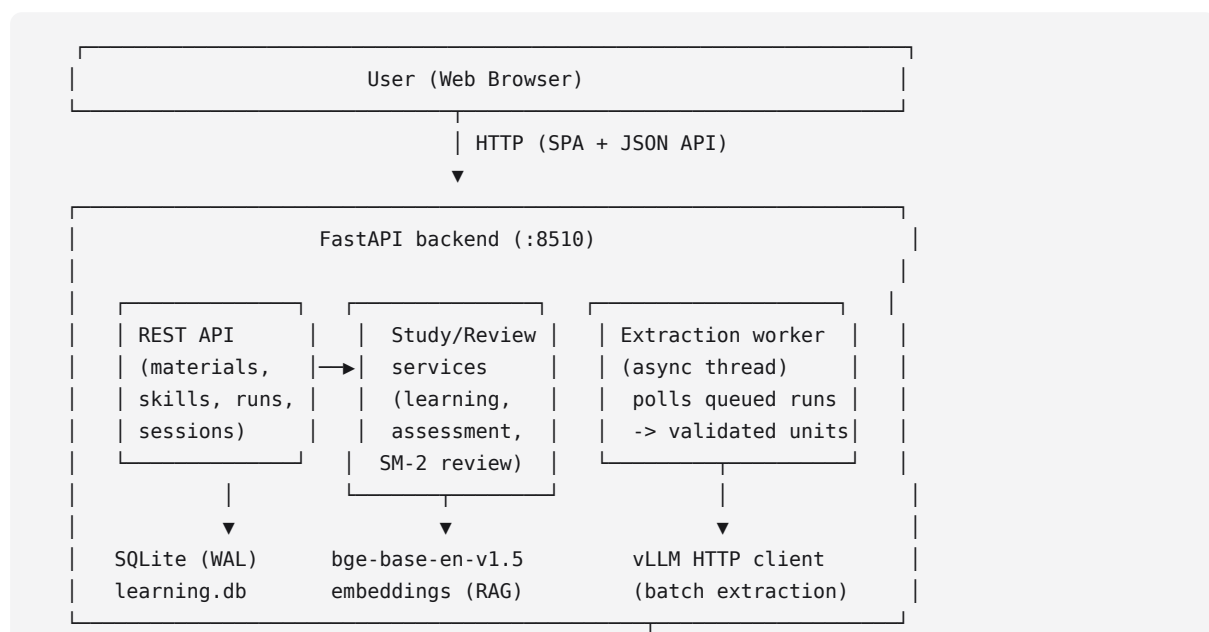
Concrete scenario used in our demo: a developer with no distributed-systems background wants to learn the **Raft consensus algorithm** from a Markdown study note. In one session they:

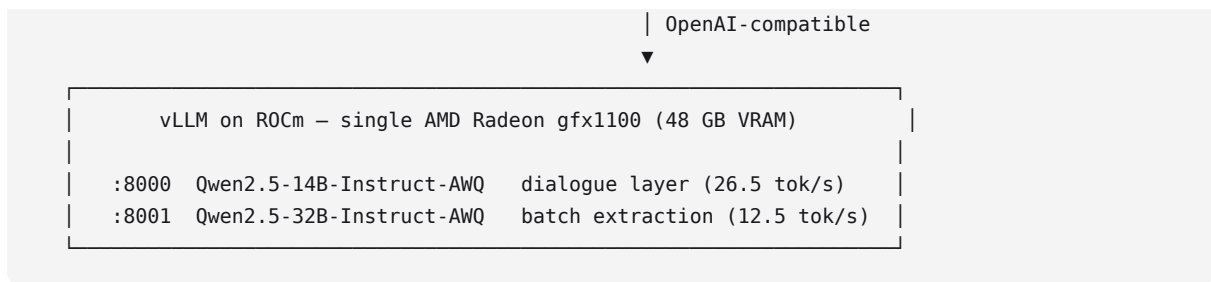
1. Upload a Markdown file — the source document is persisted verbatim.
2. Watch the local 32B model extract knowledge units, each validated against the exact source span (no fabricated content passes).
3. Browse the extracted units on the skill detail page, each traceable to its exact source sentence.
4. Learn with the Feynman technique: recall the concept in their own words, get structured feedback from the 14B dialogue model.
5. Take a first assessment — source-grounded recall, graded server-side.
6. Receive an SM-2 spaced-repetition schedule so the material is revisited at scientifically-timed intervals.

The same flow works for any topic — change the uploaded Markdown and the agent adapts. We have run the same loop on Raft, financial markets, and quantitative trading material.

Target users: self-learners, engineers preparing for interviews, and anyone who wants a private, offline study companion with real source traceability — no cloud, no data leaving the machine.

2. Agent architecture diagram





Three heavy components share the 48 GB card:

1. **Qwen2.5-14B-Instruct-AWQ** — interactive dialogue: Feynman learning, assessment grading, review feedback.
2. **Qwen2.5-32B-Instruct-AWQ** — batch extraction of knowledge units from uploaded material (higher capability, no latency requirement).
3. **BAAI/bge-base-en-v1.5** — embedding model for source grounding.

AWQ 4-bit quantization keeps both LLMs co-resident: ~12 GB (14B) + ~24 GB (32B) + embeddings = 42 GB total, leaving 6 GB headroom on the 48 GB card.

3. Core capabilities

Capability	How it works
Markdown upload	Drag & drop <code>.md</code> / <code>.markdown</code> only; original file persisted with filename; rejected otherwise (422)
Verified extraction	32B model + source validation: exact quote/offset match + field-level entailment ($LCS \geq 4$ chars)
Source traceability	Every unit stores <code>source_material</code> , <code>source_start/end</code> , <code>source_quote</code> ; viewer renders original Markdown
Feynman learning	Recall-and-explain with draft persistence (sessionStorage) and structured feedback
Mastery assessment	Server-side authoritative scoring, idempotent by payload hash
SM-2 review	Due-date scheduling on the forgetting curve; auto-resume with hidden answers
Async worker	queued → running → verified/failed; startup recovery requeues stale runs
Resilient API	drain/undrain protocol, retry (unlimited manual), delete skill/run/material
Live UI	extraction completion updates automatically (1.5 s polling, no manual refresh)

4. Model introduction & local deployment plan

All models are open-source and deployed fully locally:

- **Qwen2.5-14B-Instruct-AWQ** — dialogue layer. AWQ 4-bit, ~12 GB VRAM. Served by vLLM at `:8000` , max context 8192.
- **Qwen2.5-32B-Instruct-AWQ** — batch extraction layer. AWQ 4-bit, ~24 GB. Served by vLLM at `:8001` , max context 4096 (extraction needs no long context; the material is budgeted to ~2500 chars / 700 output tokens).
- **BAAI/bge-base-en-v1.5** — embeddings for source grounding (RAG).

Routing decision (validated by offline A/B, see `docs/evaluation/qwen2.5-vs-qwen3.json`): Qwen2.5-32B was chosen over Qwen3-32B-thinking after a real A/B on the same material. Qwen3 matched on extraction but scored worse on grading consistency; Qwen2.5-32B won the routing decision and is used for all batch extraction.

Sequential startup requirement: the two vLLM instances must be started in order (14B first, wait for readiness, then 32B). Concurrent startup can rarely cause the 14B to be mis-loaded as fp16 (27.7 GB) and fail on the shared card.

5. AMD Radeon GPU / ROCm optimization

See [docs/ROCM_OPTIMIZATION.md](#) for the full details. Summary:

- **AWQ 4-bit quantization:** $2.1\times$ faster than bf16 on gfx1100 (26.4 vs 12.3 tok/s for 14B) and halves VRAM — the enabler for dual-model co-residency on one card.
- **Dual-model co-residency:** 14B (util 0.3) + 32B (util 0.5) = 42 GB, zero throughput degradation vs solo (both 26.5 / 12.5 tok/s identical).
- **Context budgeting:** 32B max_len 4096 (batch extraction), 14B max_len 8192 (dialogue) — VRAM-aware sizing.
- **Zero cloud dependency:** all inference on local 127.0.0.1 endpoints.

6. Test coverage

- 85 backend unit tests (extraction lifecycle, source validation, entailment, study/assessment/review, drain protocol, Markdown boundary, source-document API, schema migration).
- 75 Playwright e2e tests across 375/768/1280 viewports (journeys + shell routing) on the production fallback server.
- Contract parity gate: OpenAPI snapshot $\leftrightarrow$ generated TypeScript types.
- Schema canonicalization + baseline gate with recomputed hash.